

Architecture.md

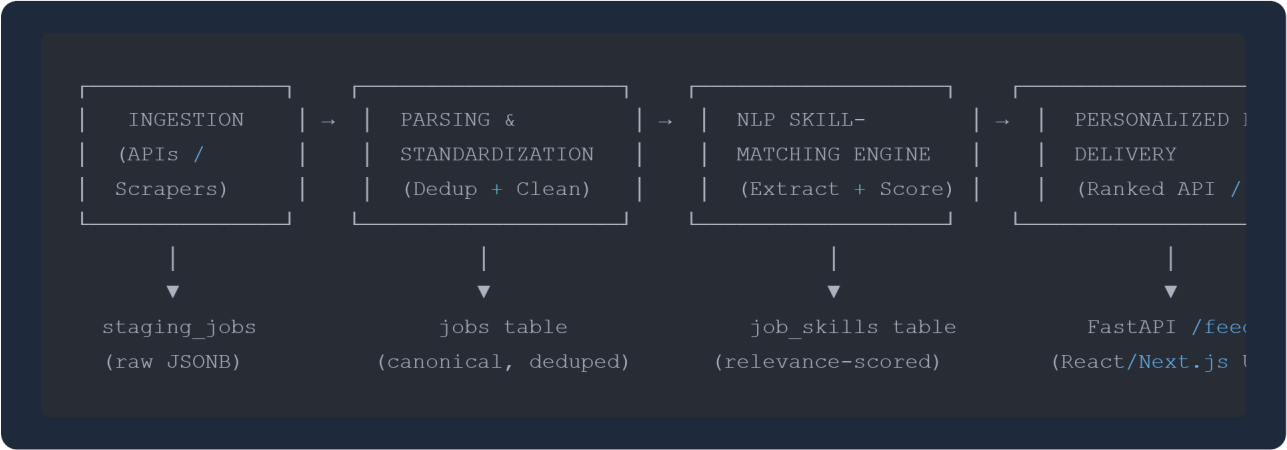
Skill-Centric Unified Job Search Aggregator System — System Architecture

1. Purpose of This Document

This document describes the **structural architecture** of the system: how it is decomposed into components, how those components communicate, what technology each is built on, and how the whole system is deployed and scaled. It is the technical companion to `design.md` (which covers schema, API, and algorithm design) and `phase.md` (which covers build sequencing).

2. Architectural Style

The system follows a **staged, unidirectional pipeline architecture**. Each stage consumes the structured output of the stage before it and produces a progressively more refined artifact. This is deliberately chosen over a monolithic request/response design because the workload is fundamentally asynchronous and I/O/CPU heterogeneous: scraping is I/O-bound and bursty, NLP extraction is CPU-bound and batchable, and feed delivery is latency-sensitive and read-heavy. Isolating these concerns lets each stage scale, fail, and redeploy independently.



3. Component Breakdown

3.1 Ingestion Layer

Responsibility: Acquire raw job posting data from ≥3 external source platforms without corrupting canonical data.

Element	Detail
Execution model	Independent, asynchronous Celery worker tasks per source platform
API-based sources	Authenticated, rate-limit-compliant scheduled HTTP polling
Scraper-based sources	<code>robots.txt</code> -compliant HTML scraping with conservative throttling
Output target	Common <code>staging_jobs</code> table (source id, raw payload, ingestion timestamp)
Failure isolation	Malformed records are quarantined in staging, not written to <code>jobs</code>
Concurrency safety	<code>UNIQUE (source_platform, source_job_id) on jobs</code> rejects duplicate inserts at the DB layer

3.2 Parsing & Standardization Layer

Responsibility: Transform heterogeneous staged payloads into the canonical `jobs` schema.

Pipeline steps (executed in order): field mapping → text normalization (case-folding, legal-suffix stripping) → location standardization (`city`, `region`, `country`) → deduplication (composite similarity heuristic + `dedup_hash` fast-path). See `design.md §4.1` for the full algorithm.

3.3 NLP Skill-Matching Engine

Responsibility: Extract structured skills from free text and score user-to-job relevance.

Phase	Trigger	Technology	Output
A — Skill Extraction	Runs asynchronously immediately after a job is standardized	spaCy custom NER + skill-taxonomy gazetteer	Rows in <code>job_skills</code> with <code>relevance_score</code>
B — Semantic Matching	Runs on feed request (or pre-computed/cached on a schedule)	scikit-learn <code>TfidfVectorizer</code> + cosine similarity	Per-user, per-job <code>match_score</code>

Decoupling Phase A from the request path is the key architectural decision that keeps the user-facing API fast: extraction is background work, matching is the only thing that touches the request path, and even that can be pre-computed.

3.4 Personalized Feed Delivery Layer

Responsibility: Expose ranked, explainable results to the end user.

Versioned REST API (FastAPI) with auto-generated OpenAPI docs.

Consumed by a React/Next.js SSR frontend.

Each feed item is annotated with overlapping and missing skills, not just a raw score — see `design.md §3`.

4. Technology Stack

Layer	Technology	Why It Was Chosen
Frontend	React + Next.js	SSR improves initial load for a data-dense feed; component model fits filterable UI
Backend / API	Python + FastAPI	Native <code>async / await</code> for concurrent scraping/API I/O; auto OpenAPI docs
Database	PostgreSQL	Relational integrity, JSONB staging, full-text search, strong indexing
NLP / Extraction	spaCy	Production-grade NER with custom entity recognizers and gazetteer matching
Semantic Matching	scikit-learn (TF-IDF + cosine similarity)	Lightweight v1 vector-space model; clear upgrade path to transformer embeddings
Task Queue	Celery + Redis (broker)	Decouples long-running scraping/NLP work from the request/response cycle
Deployment	Docker / Docker Compose	Reproducible multi-service environments across dev/staging/prod

5. Deployment Architecture

Proposed Docker Compose service topology:

```
docker-compose.yml
├─ frontend      (Next.js, port 3000)
├─ api           (FastAPI, port 8000)
├─ worker-ingest (Celery worker – scraping/API pollers, scheduled via Celery beat)
├─ worker-nlp    (Celery worker – Phase A extraction, Phase B batch scoring)
├─ postgres      (PostgreSQL 15+, persistent volume)
└─ redis         (Celery broker + result backend + feed-score cache)
```

Ingestion and NLP workers are split into separate Celery worker pools so that a burst of scraping tasks cannot starve NLP processing (or vice versa) — this is the horizontal-scaling seam referenced in `phase.md`.

Infrastructure Sizing

Environment	CPU	RAM	Storage	Notes
Development	4 cores	8 GB	20 GB SSD	Full local Compose stack (Postgres, Redis, API, workers)
Staging / PoC	2 vCPUs	4 GB	40 GB SSD	Single-instance cloud VM (e.g. AWS <code>t3.medium</code>)
Production (initial)	4 vCPUs (API) + 2 vCPUs (worker)	8–16 GB	100 GB SSD, scales with corpus	Scale ingestion workers horizontally as source count grows

6. End-to-End Data Flow (one posting's lifecycle)

A Celery ingestion task fetches a listing from Source Platform X → writes raw JSON to `staging_jobs`.

A standardization task reads from `staging_jobs` → maps fields → normalizes text/location → computes `dedup_hash`.

The `dedup_hash` is checked against existing `jobs` rows (fast exact-match path). If no exact hit, the composite fuzzy heuristic runs against recent candidates.

If unique, a new `jobs` row is inserted; if a near-duplicate, the more complete record is retained and the other's URL is linked as an alternate listing.

A Phase A extraction task runs spaCy NER against `job_description`, writing scored rows to `job_skills`.

On feed request, Phase B computes/looks up cosine similarity between the requesting user's `user_skills` vector and each candidate job's `job_skills` vector.

FastAPI returns a ranked, skill-annotated feed to the Next.js client.

7. Scalability & Reliability Considerations

Staging table as a shock absorber: upstream formatting changes never touch the canonical schema directly.

Token-bucket rate limiting per source platform, independent of internal system load.

Exponential backoff with jitter on 429 / 503 responses.

Staggered ingestion windows across sources to smooth shared infrastructure load.

Async NLP: extraction never blocks a user-facing request; only lightweight similarity scoring runs synchronously (and that is cacheable).

DB-level consistency guarantee: the `UNIQUE (source_platform, source_job_id)` constraint means duplicate-insert races between concurrent workers are rejected by Postgres itself, not by application coordination.

8. Security Considerations

Password storage via salted hashing (never plaintext), enforced at the `users.password_hash` column level.

Email format validated at the DB constraint layer as a defense-in-depth measure (application-layer validation is the primary gate).

Scraping strictly respects `robots.txt` and platform ToS; proxy rotation is treated as a resilience measure, never as a rate-limit bypass mechanism.

API authentication/session handling and secrets management are detailed in `rules.md §5`.

Companion documents: `design.md` (schema/API/algorithms), `phase.md` (delivery plan), `rules.md` (engineering rules), `memory.md` (decision log).